

CRYPTOGRAPHY – ASSESSMENT TOOL

QUESTION 3

Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.

1. TITLE

Design and Implementation of Security Association Management in IPSec

2. AIM

To design and implement a program for managing **Security Associations (SA)** in IPSec with operations such as:

1. Creation of a Security Association
2. Lookup of an existing Security Association
3. Deletion of a Security Association
4. Displaying all stored Security Associations

3. INTRODUCTION

IPSec stands for **Internet Protocol Security**. It is a collection of protocols used to provide security for communication over IP networks.

IPSec provides important security services such as:

- Confidentiality
- Data integrity
- Authentication

- Anti-replay protection

To provide these services, IPsec uses a concept called a **Security Association (SA)**.

A Security Association contains the security parameters required to process a particular flow of IP traffic.

The collection of active Security Associations is maintained in a **Security Association Database (SAD)**.

4. WHAT IS A SECURITY ASSOCIATION?

A **Security Association (SA)** is a logical relationship between two communicating entities that defines how IPsec traffic should be secured.

An SA contains information such as:

- Security Parameters Index (SPI)
- Destination IP address
- Security protocol
- Encryption algorithm
- Authentication algorithm
- Key information
- Lifetime
- Anti-replay information

An important point is that an SA is **unidirectional**.

Therefore, for two-way communication, separate SAs are normally required for the two directions.

5. SECURITY PARAMETERS INDEX (SPI)

The **Security Parameters Index (SPI)** is an identifier used to identify a particular Security Association.

Example:

SPI = 1001

When an IPsec packet arrives, the SPI helps the receiving system determine which SA should be used for processing the packet.

6. SECURITY ASSOCIATION DATABASE (SAD)

The **Security Association Database (SAD)** stores information about active Security Associations.

Example:

SPI	Destination	Protocol	Encryption	Authentication	Lifetime
1001	192.168.1.2	ESP	AES-256	HMAC-SHA256	3600 sec
1002	192.168.1.3	AH	None	HMAC-SHA256	1800 sec

The SAD allows the system to:

- Create an SA
- Find an SA
- Delete an SA

- Maintain active security information

7. IPSEC SA MANAGEMENT ARCHITECTURE

You can add the following diagram in your document:

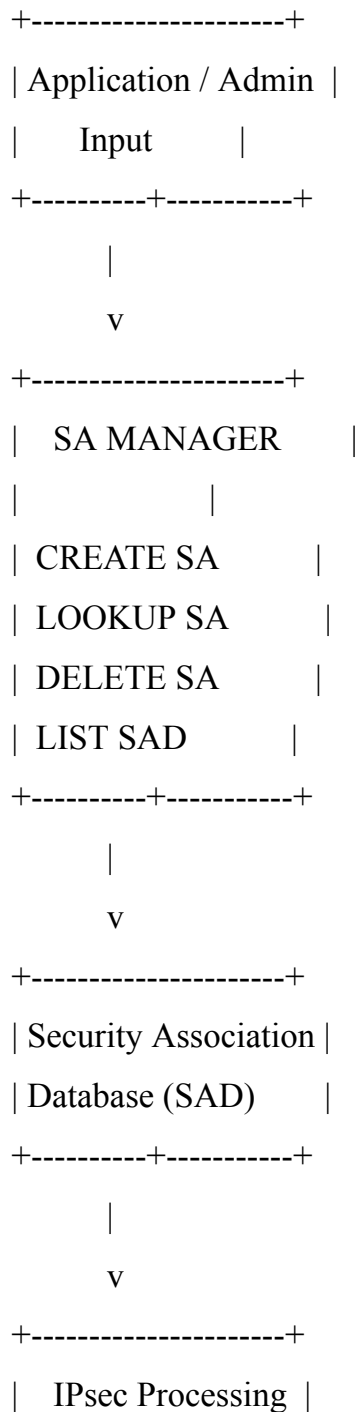




Figure 1: IPSec Security Association Management Architecture

8. SA DATA STRUCTURE

Each Security Association can be represented using the following fields:

Field	Example	Description
SPI	1001	Unique SA identifier
Destination	192.168.1.20	Destination IP address
Protocol	ESP	IPSec security protocol
Encryption	AES-256	Encryption algorithm
Authentication	HMAC-SHA256	Authentication/integrity algorithm
Lifetime	3600 seconds	Validity period

9. OPERATIONS PERFORMED

The program performs four major operations.

9.1 CREATE SA

The CREATE operation adds a new Security Association to the SAD.

Example:

SPI = 1001

Destination = 192.168.1.20

Protocol = ESP

Encryption = AES-256

Authentication = HMAC-SHA256

Lifetime = 3600

9.2 LOOKUP SA

The LOOKUP operation searches the SAD using the SPI.

Example:

Enter SPI: 1001

The program displays the corresponding SA information.

9.3 DELETE SA

The DELETE operation removes an SA from the SAD.

Example:

Enter SPI: 1001

SA deleted successfully.

9.4 LIST SAD

The LIST operation displays all currently stored Security Associations.

10. ALGORITHM

Step 1

Start the program.

Step 2

Create an empty Security Association Database.

Step 3

Display the menu:

1. CREATE SA
2. LOOKUP SA
3. DELETE SA
4. LIST SAD
5. EXIT

Step 4

For CREATE:

- Read SPI
- Read destination IP
- Read protocol
- Read encryption algorithm
- Read authentication algorithm
- Read lifetime
- Store the information in the SAD

Step 5

For LOOKUP:

- Read SPI
- Search the SAD
- Display the corresponding SA

Step 6

For DELETE:

- Read SPI
- Search the SAD
- Delete the corresponding SA

Step 7

For LIST:

- Display all stored Security Associations

Step 8

Repeat the process until EXIT is selected.

Step 9

Stop the program.

11. FLOWCHART

START

|

v

Initialize SAD

|

v

Display Menu

|

+-----+-----+

| | |

v v v

CREATE LOOKUP DELETE

| | |

v v v

Add SA Find SA Remove SA

| | |

+-----+-----+

|

v

LIST SAD

|

v

Display Records

|

v

Exit Selected?

/ \

No Yes

| |

+----> Menu v

STOP

12. PYTHON PROGRAM

```
class SecurityAssociationManager:
```

```
    def __init__(self):
```

```
        self.sad = {}
```

```
    def create_sa(self, spi, destination, protocol,  
                  encryption, authentication, lifetime):
```

```
        if spi in self.sad:
```

```
            print("Error: SPI already exists.")
```

```
            return
```

```
        self.sad[spi] = {
```

```
            "destination": destination,
```

```
            "protocol": protocol,
```

```
            "encryption": encryption,
```

```
            "authentication": authentication,
```

```
            "lifetime": lifetime
```

```
        }
```

```
        print("SA created successfully.")
```

```
    def lookup_sa(self, spi):
```

```
        sa = self.sad.get(spi)
```

```
        if sa is None:
```

```
print("SA not found.")  
return
```

```
print("\n--- SA DETAILS ---")  
print("SPI:", spi)
```

```
for key, value in sa.items():  
    print(f'{key.title()}: {value}')
```

```
def delete_sa(self, spi):
```

```
    if spi in self.sad:  
        del self.sad[spi]  
        print("SA deleted successfully.")  
    else:  
        print("SA not found.")
```

```
def list_sas(self):
```

```
    if not self.sad:  
        print("SAD is empty.")  
        return
```

```
print("\n--- SECURITY ASSOCIATION DATABASE ---")
```

```
for spi, sa in self.sad.items():
```

```
    print(  
        f"SPI={spi} | "
```

```
        f'{sa['destination']}' | "  
        f'{sa['protocol']}' | "  
        f'{sa['encryption']}'"  
    )
```

```
def main():
```

```
    manager = SecurityAssociationManager()
```

```
    while True:
```

```
        print("\n1. CREATE SA")
```

```
        print("2. LOOKUP SA")
```

```
        print("3. DELETE SA")
```

```
        print("4. LIST SAD")
```

```
        print("5. EXIT")
```

```
        choice = input("Enter choice: ")
```

```
        if choice == "1":
```

```
            spi = int(input("Enter SPI: "))
```

```
            destination = input("Enter destination IP: ")
```

```
            protocol = input("Enter protocol (AH/ESP): ")
```

13. SAMPLE OUTPUT

1. CREATE SA
2. LOOKUP SA
3. DELETE SA
4. LIST SAD
5. EXIT

Enter choice: 1

Enter SPI: 1001

Enter destination IP: 192.168.1.20

Enter protocol (AH/ESP): ESP

Enter encryption algorithm: AES-256

Enter authentication algorithm: HMAC-SHA256

Enter lifetime in seconds: 3600

SA created successfully.

Lookup Operation

Enter choice: 2

Enter SPI to lookup: 1001

--- SA DETAILS ---

SPI: 1001

Destination: 192.168.1.20

Protocol: ESP

Encryption: AES-256

Authentication: HMAC-SHA256

Lifetime: 3600

Delete Operation

Enter choice: 3

Enter SPI to delete: 1001

SA deleted successfully.

14. COMPLEXITY ANALYSIS

The program uses a Python dictionary to implement the SAD.

Operation	Average Time Complexity
------------------	--------------------------------

Create SA	$O(1)$
-----------	--------

Lookup SA	$O(1)$
--------------	--------

Delete SA	$O(1)$
-----------	--------

List all SAs	$O(n)$
-----------------	--------

Where **n** is the number of Security Associations stored in the database.

15. RESULT

Thus, a Python program for managing **IPSec Security Associations** was successfully designed and implemented.

The program successfully performs:

- Security Association creation
- Security Association lookup
- Security Association deletion
- Security Association database listing

The program demonstrates how an SAD can maintain security-related information required for IPSec processing.

QUESTION 4

Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.

16. TITLE

Design and Implementation of a Simplified PGP System for Secure Email Communication

17. AIM

To implement a simplified **Pretty Good Privacy (PGP)** system that provides secure email communication using:

- Public key encryption
- Symmetric session-key encryption

- SHA-256 hashing
- Digital signatures
- Message decryption
- Signature verification

18. INTRODUCTION

Pretty Good Privacy (PGP) is a security technology used to protect electronic communication.

PGP can provide:

- Confidentiality
- Authentication
- Integrity
- Non-repudiation

PGP uses a combination of **symmetric cryptography** and **public-key cryptography**.

This combination is called **hybrid encryption**.

19. WHY HYBRID ENCRYPTION IS USED

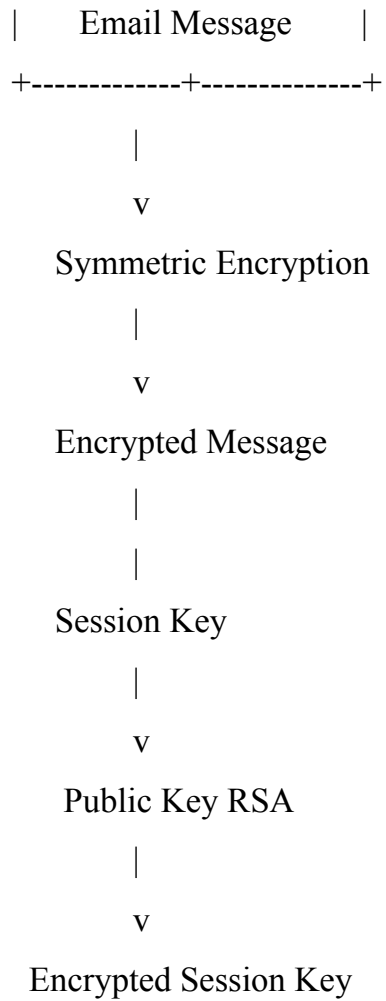
Symmetric encryption is very fast and is suitable for encrypting large amounts of data.

Public-key encryption is computationally more expensive but is useful for securely exchanging keys.

Therefore, PGP combines both methods.

PGP HYBRID ENCRYPTION

+-----+



20. MAIN COMPONENTS OF PGP

20.1 Plaintext

The original email message written by the sender.

Example:

Meet me at 10 AM for the project discussion.

20.2 Session Key

A random temporary key generated for encrypting the message.

The session key is used only for the particular communication.

20.3 Symmetric Encryption

The session key is used to encrypt the actual email message.

In this classroom demonstration, a simple XOR-based encryption mechanism is used.

Important: XOR here is only for demonstrating the concept and is not secure production cryptography.

21. PUBLIC KEY ENCRYPTION

Public-key cryptography uses two keys:

Public Key

Can be shared with everyone.

Private Key

Must be kept secret by the owner.

For the receiver:

Receiver Public Key

|

v

Encrypt Session Key

|

v

Encrypted Session Key

The receiver uses the corresponding private key to recover the protected session-key value.

22. DIGITAL SIGNATURE

A digital signature is used to verify:

- Who sent the message
- Whether the message was modified

The simplified process is:

Original Message

|

v

SHA-256

|

v

Message Hash

|

v

Sender Private Key

|

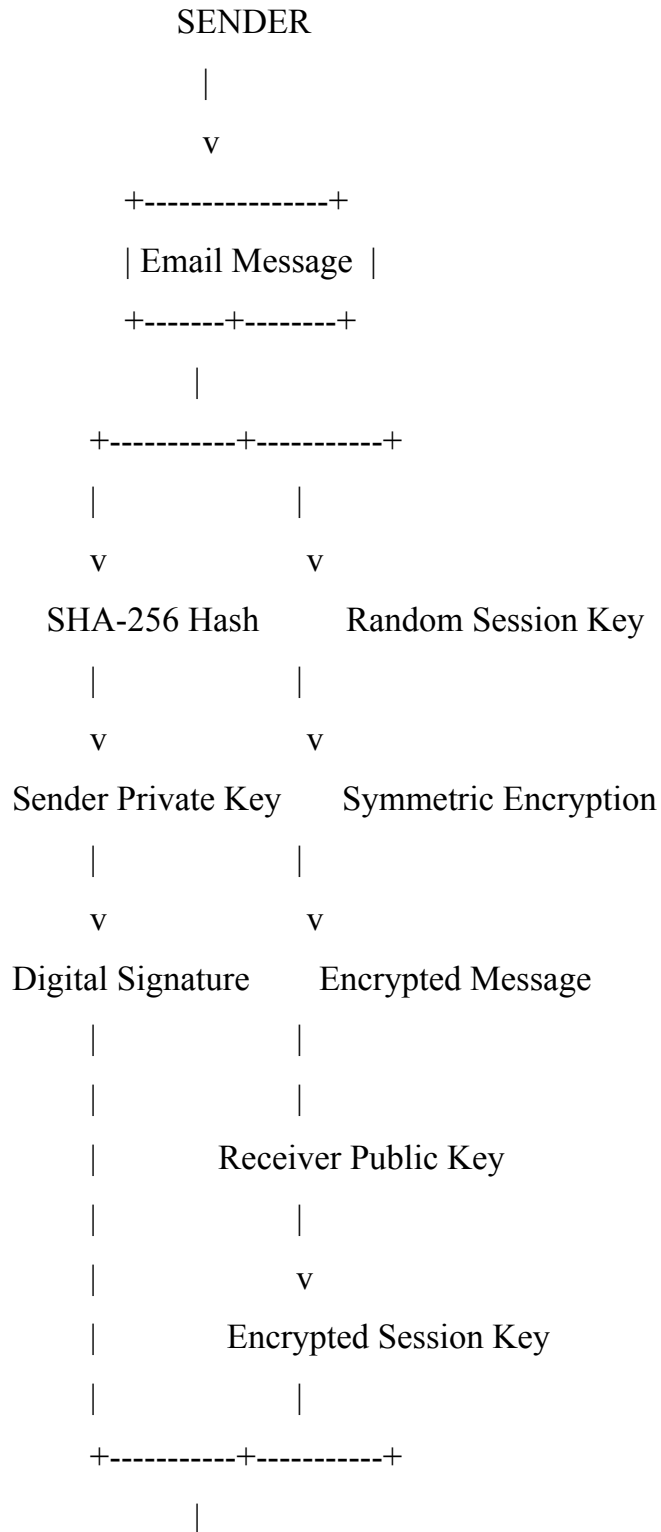
v

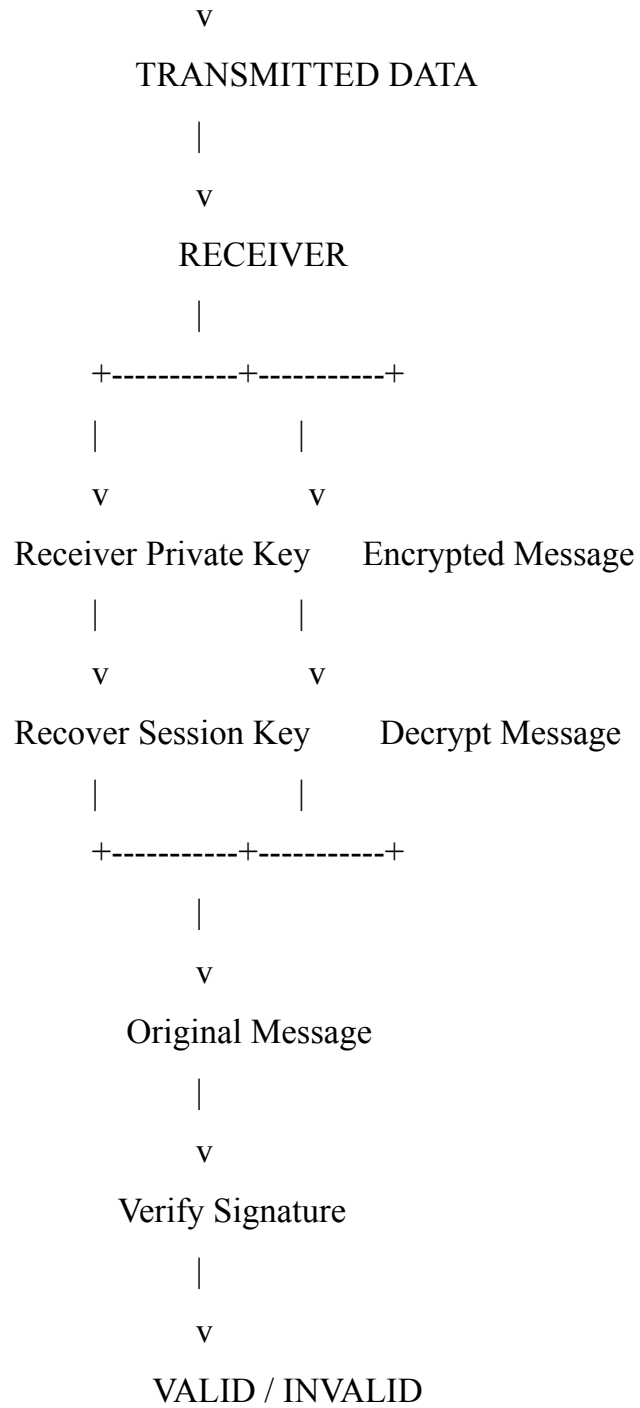
Digital Signature

The receiver verifies the signature using the sender's public key.

23. PGP SECURITY FLOW

Use this as the main diagram:





24. PGP WORKING PROCESS

Step 1 – Enter Message

The sender enters the email message.

Example:

Meet me at 10 AM.

Step 2 – Generate Session Key

A random session key is generated.

Session Key

|

v

Random temporary key

Step 3 – Encrypt Message

The email message is encrypted using the session key.

Plaintext

+

Session Key

|

v

Encrypted Message

Step 4 – Generate Hash

SHA-256 is applied to the original message.

Message

|

v

SHA-256

|

v

Hash / Digest

Step 5 – Create Digital Signature

The sender's private key is used to create the digital signature.

Hash

|

v

Sender Private Key

|

v

Digital Signature

Step 6 – Encrypt Session Key

The receiver's public key is used to protect the session key.

Session Key

|

v

Receiver Public Key

|

v

Encrypted Session Key

Step 7 – Send the Package

The sender transmits:

Encrypted Message

+

Encrypted Session Key

+

Digital Signature

Step 8 – Receiver Decryption

The receiver uses their private key to recover the protected session-key value.

Step 9 – Decrypt Message

The recovered session key is used to decrypt the encrypted message.

Step 10 – Verify Signature

The receiver verifies the digital signature using the sender's public key.

If the signature is valid:

Signature Valid = TRUE

Otherwise:

Signature Valid = FALSE

25. ALGORITHM

Sender Side

1. Start the program.
2. Generate sender RSA key pair.
3. Generate receiver RSA key pair.
4. Enter the email message.
5. Generate a random session key.
6. Encrypt the message using the session key.
7. Calculate SHA-256 hash of the message.
8. Generate a digital signature using the sender private key.
9. Encrypt/protect the session key using the receiver public key.
10. Send encrypted message, encrypted session key and signature.

Receiver Side

11. Receive the encrypted package.
12. Use receiver private key to recover the protected session-key value.
13. Decrypt the message.
14. Calculate/verify the message hash.
15. Verify the digital signature using sender public key.
16. Display the decrypted message.
17. Display whether the signature is valid.
18. Stop the program.

26. PYTHON PROGRAM

```
import hashlib
import secrets
```

```
# ----- RSA Utilities -----
```

```
def gcd(a, b):
```

```
    while b:
```

```
        a, b = b, a % b
```

```
    return a
```

```
def egcd(a, b):
```

```
    if b == 0:
```

```
        return a, 1, 0
```

```
    g, x1, y1 = egcd(b, a % b)
```

```
    return g, y1, x1 - (a // b) * y1
```

```
def mod_inverse(a, m):
```

```
    g, x, _ = egcd(a, m)
```

```
    if g != 1:
```

```
        raise ValueError("Inverse does not exist")
```

```
    return x % m
```

```
def is_prime(n):
```

```
if n < 2:  
    return False
```

```
if n % 2 == 0:  
    return n == 2
```

```
p = 3
```

```
while p * p <= n:
```

```
    if n % p == 0:  
        return False
```

```
    p += 2
```

```
return True
```

```
def generate_prime():
```

```
    while True:
```

```
        n = secrets.randbelow(400) + 200
```

```
        if n % 2 == 0:  
            n += 1
```

```
        if is_prime(n):
```

```
return n
```

```
def generate_rsa_keys():
```

```
    p = generate_prime()
```

```
    q = generate_prime()
```

```
    while q == p:
```

```
        q = generate_prime()
```

27. SAMPLE INPUT

Enter email message:

Meet me at 10 AM for the project discussion.

28. SAMPLE OUTPUT

===== SIMPLIFIED PGP SECURE EMAIL =====

--- SENDER SIDE ---

Original message:

Meet me at 10 AM for the project discussion.

Encrypted message:

4c8a91f7b2319c4a...

Encrypted session key:

731245

Digital signature:

183924

--- RECEIVER SIDE ---

Recovered RSA session value:

526781

Decrypted message:

Meet me at 10 AM for the project discussion.

Signature valid:

True

PGP verification successful.

Note: The encrypted values will be different every time because random keys are generated.

29. EXPLANATION OF THE PROGRAM

generate_prime()

This function generates a prime number for the educational RSA implementation.

```
def generate_prime():
```

Prime numbers are required for RSA key generation.

generate_rsa_keys()

This function generates:

Public Key

Private Key

The public key can be shared, while the private key must remain secret.

xor_encrypt()

This function performs the simplified symmetric encryption of the email message.

```
encrypted_message = xor_encrypt(  
    message,  
    session_key  
)
```

sha256_int()

This function generates a SHA-256 hash of the message.

```
digest = hashlib.sha256(  
    text.encode("utf-8")  
)
```

The hash represents the contents of the message.

sign_message()

This function generates a digital signature using the sender's private key.

Message

↓

SHA-256

↓

Hash



Sender Private Key



Signature

verify_signature()

The receiver uses the sender's public key to verify the signature.

If the original message has not been modified:

True

is returned.

If the message has been modified:

False

is returned.

30. SECURITY SERVICES PROVIDED

Security Service	Technique Used
Confidentiality	Symmetric encryption
Key Protection	RSA public-key encryption
Integrity	SHA-256
Authentication	Digital signature

Message	Public-key signature
Verification	verification

31. ADVANTAGES OF PGP

1. Confidentiality

Unauthorized users cannot easily read the encrypted message.

2. Authentication

The receiver can verify the identity of the sender through the digital signature.

3. Integrity

Any modification to the message can cause signature verification to fail.

4. Hybrid Encryption

Combining symmetric and asymmetric cryptography provides a practical security architecture.

5. Secure Key Exchange

Public-key cryptography can be used to protect session keys.

32. LIMITATIONS OF THIS SIMPLIFIED IMPLEMENTATION

This program is designed for **academic demonstration only**.

The following limitations exist:

1. The RSA implementation uses very small educational prime numbers.

2. The XOR encryption used here is not a secure modern cipher.
3. The program does not implement the complete OpenPGP packet format.
4. It does not implement a real PGP key ring.
5. It does not implement certificate management.
6. It does not implement a trust network.
7. It does not provide production-level authenticated encryption.
8. Real-world applications should use established cryptographic libraries.

Therefore, this program demonstrates the **concept and workflow of PGP**, rather than implementing production-grade OpenPGP.

33. COMPARISON BETWEEN IPSEC SA AND PGP

Feature	IPSec SA	PGP
Main purpose	Secure IP communication	Secure messages/email
Main concept	Security Association	Key-based message protection
Database	SAD	Key management system
Identifier	SPI	Public/private key
Encryption	AH/ESP-related algorithms	Symmetric + public key
Authentication	IPsec authentication mechanisms	Digital signature
Integrity	Authentication/integrity mechanisms	Hash + digital signature
Communication level	Network layer	Message/application level

Main operation

Create, lookup, delete SA

Encrypt, decrypt, sign,
verify

34. REAL-WORLD APPLICATIONS

IPSec

IPSec is commonly used for:

- VPNs
- Site-to-site VPN connections
- Remote-access VPNs
- Secure corporate networks
- Secure IP communication

PGP

PGP/OpenPGP concepts are used for:

- Secure email
- Encrypted files
- Digital signatures
- Secure document exchange
- Software/package signing
- Confidential communication